Predictive Modeling and Risk Scoring for Bank Customer Churn

1. Project Overview

Title

Predictive Modeling and Risk Scoring for Bank Customer Churn

Domain

Banking Analytics / Customer Intelligence / Predictive Analytics

Industry Importance

Customer churn is one of the most critical problems in retail banking because acquiring a new customer costs significantly more than retaining an existing one.

Banks lose revenue when customers:

- Close accounts
- Shift to competitors
- Reduce product usage
- Stop engaging with banking services

Modern banking systems therefore require:

- Early churn detection
- Predictive intelligence
- Risk segmentation
- Personalized retention systems

This project develops:

- A machine learning churn prediction system
- Customer-level risk scoring

- Explainable AI framework
- Interactive Streamlit dashboard

2. Business Understanding

Why Customer Churn Matters

Customer churn affects:

Revenue Stability

Loss of customers reduces:

- Deposits
- Loan opportunities
- Transaction revenue
- Investment product sales

Customer Lifetime Value (CLV)

Long-term profitable customers generate:

- Recurring revenue
- Cross-selling opportunities
- Referrals

Competitive Position

High churn rates indicate:

- Poor engagement
- Weak loyalty
- Better competitor offerings

3. Problem Statement

Banks possess extensive customer-level data but often fail to transform it into predictive intelligence.

Existing Challenges

Banks lack:

- Accurate churn prediction models
- Quantitative risk scoring systems
- Explainable customer behavior insights

Current Retention Problems

Retention strategies become:

- Reactive instead of proactive
- Expensive and inefficient
- Broad rather than personalized

4. Project Objectives

Primary Objectives

1. Predict Customer Churn

Develop ML models capable of identifying customers likely to leave.

2. Generate Risk Scores

Assign churn probability scores between:

$$0 \rightarrow 1$$

Where:

- 0 = very low risk
- 1 = extremely high risk

3. Identify Churn Drivers

Determine which features contribute most toward churn.

Secondary Objectives

Reduce False Positives

Avoid unnecessarily targeting loyal customers.

Improve Interpretability

Enable banking managers to understand predictions.

Scenario-Based Risk Analysis

Simulate customer behavior changes and observe risk variations.

5. Dataset Description

Column	Description
CustomerId	Unique customer ID
Surname	Customer surname
CreditScore	Creditworthiness score
Geography	Country (France, Spain, Germany)
Gender	Male/Female
Age	Customer age
Tenure	Years with bank
Balance	Bank account balance
NumOfProducts	Number of products used
HasCrCard	Credit card ownership
IsActiveMember	Customer activity status
EstimatedSalary	Annual salary
Exited	Churn target variable

6. Machine Learning Workflow

Step 1 — Data Collection

Dataset sources:

- Kaggle Bank Churn Dataset
- ECB open banking datasets
- Simulated banking data

Typical dataset size:

- 10,000+ records
- 10–15 features

Step 2 — Data Preprocessing

A. Data Inspection

Check:

- Shape of dataset
- Data types
- Missing values
- Duplicate rows

Example

```
df.shape
df.info()
df.isnull().sum()
```

B. Remove Non-Informative Features

These columns do not contribute to prediction:

```
df.drop(['CustomerId', 'Surname'], axis=1, inplace=True)
```

Reason:

- IDs contain no behavioral pattern
- Names do not affect churn

C. Handle Missing Values

Possible techniques:

- Mean imputation
- Median imputation
- Mode imputation

Example:

df.fillna(df.median(), inplace=True)

D. Encode Categorical Variables

One-Hot Encoding

Convert categories into binary columns.

df = pd.get_dummies(df, columns=['Geography', 'Gender'], drop_first=True)

Generated features:

- Geography_Germany
- Geography_Spain
- Gender_Male

E. Feature Scaling

Required for:

- Logistic Regression
- Gradient models

Use:

- StandardScaler
- MinMaxScaler

from sklearn.preprocessing import StandardScaler

7. Feature Engineering

Feature engineering improves predictive power.

A. Balance-to-Salary Ratio

Measures financial dependence.

$$\text{BalanceSalaryRatio} = \frac{\text{Balance}}{\text{EstimatedSalary}}$$

Interpretation

High ratio may indicate:

- Financial stress
- Loan dependency
- Increased churn risk

B. Product Density Indicator

Measures customer product engagement.

$$\text{ProductDensity} = \frac{\text{NumOfProducts}}{\text{Tenure} + 1}$$

Interpretation

Low product density may indicate:

- Weak banking relationship
- Lower loyalty

C. Engagement-Product Interaction

$$\text{EngagementInteraction} = \text{IsActiveMember} \times \text{NumOfProducts}$$

Importance

Captures:

- Behavioral engagement
- Product utilization

D. Age-Tenure Interaction

$$\text{AgeTenureInteraction} = \text{Age} \times \text{Tenure}$$

Business Meaning

Long-term older customers are often:

- More stable
- Lower churn risk

8. Exploratory Data Analysis (EDA)

Important Visualizations

1. Churn Distribution

Shows class imbalance.

Insights:

- % retained customers
- % churned customers

2. Churn by Geography

Compare churn rates across:

- Germany
- France
- Spain

Possible insight:
Germany often shows higher churn.

3. Age vs Churn

Older customers may churn more frequently.

4. Balance Distribution

High-balance customers sometimes show higher exit rates.

5. Active Membership vs Churn

Inactive members typically churn more.

6. Number of Products vs Churn

Customers with:

- 1 product → higher churn
- 3–4 products → lower churn

9. Train-Test Strategy

Stratified Split

Preserves churn distribution.

from sklearn.model_selection import train_test_split

Example:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

Why Stratification Matters

Without stratification:

- Minority class imbalance increases
- Model bias occurs

10. Model Development

A. Logistic Regression

Purpose

Baseline interpretable model.

Advantages

- Easy to explain
- Regulatory-friendly
- Fast training

Weakness

Cannot capture complex nonlinear relationships.

Logistic Regression Formula

$$P(Y = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n)}}$$

B. Decision Tree

Advantages

- Easy interpretation
- Handles nonlinear patterns

Weakness

- Overfitting risk

C. Random Forest

Concept

Ensemble of decision trees.

Benefits

- Better generalization
- Reduced overfitting
- Strong feature importance analysis

D. Gradient Boosting

Working Principle

Sequential learning from previous errors.

Advantages

- High accuracy
- Handles complex patterns

E. XGBoost (Advanced)

Why XGBoost?

Industry-standard boosting algorithm.

Features

- Regularization
- Parallel processing
- Missing value handling
- Superior performance

11. Model Evaluation Metrics

A. Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Measures

Overall prediction correctness.

B. Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Importance

Controls false churn alarms.

C. Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

Importance

Captures actual churners.

D. F1-Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Importance

Balances precision and recall.

E. ROC-AUC

Measures:

- Model discrimination ability
- Probability ranking quality

AUC interpretation:

- 0.5 = random
- 0.7–0.8 = good
- 0.8–0.9 = excellent

12. Confusion Matrix

	Predicted No Churn	**Predicted Churn**
Actual No Churn	TN	FP
Actual Churn	FN	TP

Banking Interpretation

False Positive (FP)

Customer predicted to churn but stays.

Cost:

- Unnecessary retention incentives

False Negative (FN)

Customer predicted safe but actually leaves.

Cost:

- Revenue loss
- Missed retention opportunity

Banks usually prioritize:

- Higher Recall
- Lower FN

13. Churn Risk Scoring System

Risk Categories

Probability	**Risk Level**
0.00–0.30	Low Risk
0.31–0.60	Medium Risk
0.61–1.00	High Risk

Example

Customer	**Probability**	**Risk**
A	0.12	Low

Customer	Probability	Risk
B	0.48	Medium
C	0.87	High

14. Explainable AI (XAI)

Banking models must be explainable due to:

- Regulatory compliance
- Transparency requirements
- Ethical AI standards

A. Feature Importance

Top churn drivers often include:

- Age
- Balance
- Activity status
- Number of products

B. SHAP Values

SHAP explains:

- Why a prediction occurred
- Feature contribution magnitude

Example Interpretation

A churn score increases because:

- High balance
- Low activity

- Single product ownership

C. Partial Dependence Plots

Show how churn changes with:

- Age
- Balance
- Product count

15. Business Insights

Common Banking Churn Insights

1. Inactive Customers Churn More

Low engagement strongly predicts churn.

2. Customers with Fewer Products Are Less Loyal

Cross-selling improves retention.

3. Middle-Aged Customers Often Show Higher Churn

Competitive product switching increases.

4. High-Balance Customers Require Special Attention

They contribute significant revenue

16. Business Recommendations

A. Proactive Retention Campaigns

Target:

- High-risk customers

- Customers with declining engagement

Strategies:

- Personalized offers
- Loyalty rewards
- Fee waivers

B. Product Bundling

Encourage:

- Savings + credit card
- Loans + insurance

More products increase stickiness.

C. Engagement Enhancement

Improve:

- Mobile app experience
- Financial education
- Customer support responsiveness

D. High-Value Customer Protection

Special handling for:

- Premium accounts
- High-balance customers

17. Streamlit Dashboard Architecture

Dashboard Modules

A. Customer Risk Calculator

Inputs

- Age

- Geography
- Balance
- Activity status
- Products

Output

- Churn probability
- Risk category

B. Probability Visualization

Charts:

- Gauge charts
- Risk distribution histograms
- ROC curves

C. Feature Importance Dashboard

Displays:

- SHAP importance
- Top churn drivers

D. What-If Simulator

Allows users to:

- Change activity status
- Add products
- Increase engagement

Observe probability changes instantly.

18. Streamlit Workflow

Step 1

Load trained model.

```
import joblib
model = joblib.load('model.pkl')
```

Step 2

Collect user input.

```
st.number_input()
st.selectbox()
```

Step 3

Predict probability.

```
prediction = model.predict_proba(input_data)
```

Step 4

Display risk category.

```
if prob > 0.6:
   st.error("High Churn Risk")
```

19. Project Folder Structure

```
Bank-Churn-Prediction/
│
├── data/
├── notebooks/
├── models/
├── streamlit_app/
├── reports/
├── visuals/
├── README.md
├── requirements.txt
└── app.py
```

20. Research Paper Structure

Abstract

Overview of:

- Problem
- Methodology
- Results

Introduction

Discuss:

- Banking churn challenges
- Importance of predictive analytics

Literature Review

Compare:

- Previous churn models
- Existing banking analytics approaches

Methodology

Explain:

- Data preprocessing
- Feature engineering
- ML algorithms

Results

Include:

- Accuracy
- ROC-AUC
- Confusion matrix
- SHAP analysis

Conclusion

Summarize:

- Key findings
- Business value
- Future improvements

21. Executive Summary for Stakeholders

The proposed churn intelligence framework enables banks to transition from reactive customer management to proactive retention optimization.

The system:

- Predicts churn risk early
- Prioritizes high-risk customers
- Improves retention efficiency
- Supports explainable decision-making

Expected benefits include:

- Reduced customer attrition
- Higher profitability
- Improved customer satisfaction
- Better resource allocation

22. Advanced Enhancements

Future Improvements

Deep Learning Models

- ANN
- LSTM behavioral analysis

Real-Time Risk Scoring

Streaming customer analytics.

Recommendation Systems

Personalized retention recommendations.

Behavioral Sequence Analysis

Analyze transaction timelines.

23. Expected Outcomes

The final system should achieve:

- High predictive performance
- Explainable AI outputs
- Business-friendly insights
- Interactive deployment

24. Final Conclusion

This project transforms customer churn analysis into a proactive intelligence system capable of predicting customer exits before they occur. By integrating machine learning, feature engineering, explainable AI, and risk scoring, the framework enables banks to optimize retention strategies, reduce revenue loss, and improve customer relationship management.

The emphasis on engagement behavior, product utilization, and explainable predictions makes the solution both operationally effective and strategically valuable for modern banking institutions.

Column	Meaning	Example	Explanation
Customer Id	Unique customer number	15634602	Used for identification only
Surname	Customer last name	Hargrave	Personal identification field
Credit Score	Creditworthiness score	619	Lower scores may increase churn risk
Geography	Customer country	France	Region-based churn behavior
Gender	Male/Female	Female	Demographic feature
Age	Customer age	42	Strong churn predictor
Tenure	Years with bank	2	Indicates loyalty duration
Balance	Account balance	125510.82	High-value customers are important
Num Of Products	Products used	3	More products → stronger engagement
Has Cr Card	Credit card ownership	1	1 = Yes, 0 = No
Is Active Member	Customer activity status	1	Active users churn less
Estimated Salary	Annual salary estimate	101348.88	Financial capability indicator
Exited	Churn status	1	1 = Customer left bank

Coding's

```
# BANK CUSTOMER CHURN PREDICTION PROJECT

# Predictive Modeling and Risk Scoring


# ==========================
# 1. Import Libraries
# ==========================


import pandas as pd
import numpy as np


# Visualization
import matplotlib.pyplot as plt
import seaborn as sns


# Preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler


# Models
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier


# Evaluation
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    roc_curve
)


# Ignore warnings
import warnings
warnings.filterwarnings('ignore')
```

```
# 2. Load Dataset


# Replace with your file path

df = pd.read_csv("Churn_Modelling.csv")


#
============================================================

# 3. Basic Data Inspection

#
============================================================


print("Dataset Shape:")

print(df.shape)


print("\nFirst 5 Rows:")

print(df.head())


print("\nDataset Information:")

print(df.info())


print("\nMissing Values:")

print(df.isnull().sum())
```

```
print("\nDuplicate Rows:")

print(df.duplicated().sum())


#
================================================================

# 4. Drop Non-Informative Features

#
================================================================


df.drop(['CustomerId', 'Surname'], axis=1, inplace=True)


#
================================================================

# 5. Feature Engineering

#
================================================================


# A. Balance-to-Salary Ratio

df['BalanceSalaryRatio'] = df['Balance'] / (df['EstimatedSalary'] + 1)


# B. Product Density

df['ProductDensity'] = df['NumOfProducts'] / (df['Tenure'] + 1)
```

```
# C. Engagement Interaction
df['EngagementInteraction'] = (
    df['IsActiveMember'] * df['NumOfProducts']
)

# D. Age-Tenure Interaction
df['AgeTenureInteraction'] = (
    df['Age'] * df['Tenure']
)

#
# 6. Encode Categorical Variables
#

df = pd.get_dummies(
    df,
    columns=['Geography', 'Gender'],
    drop_first=True
)
```

```
#
# ============================================================
# 7. Exploratory Data Analysis (EDA)
#
# ============================================================

# ---------------------------
# Churn Distribution
# ---------------------------

plt.figure(figsize=(6,5))
sns.countplot(x='Exited', data=df)
plt.title("Churn Distribution")
plt.show()

# ---------------------------
# Age Distribution by Churn
# ---------------------------

plt.figure(figsize=(8,5))
sns.histplot(
```

```
    data=df,
    x='Age',
    hue='Exited',
    kde=True,
    bins=30
)
plt.title("Age Distribution by Churn")
plt.show()

# --------------------------
# Balance Distribution
# --------------------------

plt.figure(figsize=(8,5))
sns.histplot(
    data=df,
    x='Balance',
    hue='Exited',
    kde=True
)
plt.title("Balance Distribution by Churn")
plt.show()
```

```
# -------------------------
# Products vs Churn
# -------------------------

plt.figure(figsize=(7,5))
sns.countplot(
    x='NumOfProducts',
    hue='Exited',
    data=df
)
plt.title("Products vs Churn")
plt.show()

#
# 8. Split Features and Target
#

X = df.drop('Exited', axis=1)
y = df['Exited']
```

```
#
# ============================================================
# 9. Train-Test Split
#
# ============================================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    stratify=y,
    random_state=42
)

#
# ============================================================
# 10. Feature Scaling
#
# ============================================================

scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


#
============================================================

# 11. Logistic Regression Model

#
============================================================


log_model = LogisticRegression()


log_model.fit(X_train_scaled, y_train)


y_pred_log = log_model.predict(X_test_scaled)

y_prob_log = log_model.predict_proba(X_test_scaled)[:,1]


#
============================================================

# 12. Decision Tree Model

#
============================================================


dt_model = DecisionTreeClassifier(
```

```
    max_depth=5,
    random_state=42
)

dt_model.fit(X_train, y_train)

y_pred_dt = dt_model.predict(X_test)
y_prob_dt = dt_model.predict_proba(X_test)[:,1]

#
# 13. Random Forest Model
#

rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=7,
    random_state=42
)

rf_model.fit(X_train, y_train)
```

y_pred_rf = rf_model.predict(X_test)

y_prob_rf = rf_model.predict_proba(X_test)[:,1]

#
==========

14. Gradient Boosting Model

#
==========

gb_model = GradientBoostingClassifier()

gb_model.fit(X_train, y_train)

y_pred_gb = gb_model.predict(X_test)

y_prob_gb = gb_model.predict_proba(X_test)[:,1]

#
==========

15. Model Evaluation Function

#
==========

```
def evaluate_model(y_test, y_pred, y_prob, model_name):

    print(f"\n==============================")
    print(f"{model_name}")
    print(f"==============================")

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    roc_auc = roc_auc_score(y_test, y_prob)

    print(f"Accuracy  : {accuracy:.4f}")
    print(f"Precision : {precision:.4f}")
    print(f"Recall    : {recall:.4f}")
    print(f"F1 Score  : {f1:.4f}")
    print(f"ROC-AUC   : {roc_auc:.4f}")

    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))
```

```
#
# ============================================================
# 16. Evaluate All Models
#
# ============================================================

evaluate_model(
    y_test,
    y_pred_log,
    y_prob_log,
    "Logistic Regression"
)

evaluate_model(
    y_test,
    y_pred_dt,
    y_prob_dt,
    "Decision Tree"
)

evaluate_model(
    y_test,
```

```
    y_pred_rf,
    y_prob_rf,
    "Random Forest"
)

evaluate_model(
    y_test,
    y_pred_gb,
    y_prob_gb,
    "Gradient Boosting"
)

#
# 17. Confusion Matrix
#

cm = confusion_matrix(y_test, y_pred_rf)

plt.figure(figsize=(6,5))
sns.heatmap(
```

```
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

#
# 18. ROC Curve
#

fpr, tpr, thresholds = roc_curve(y_test, y_prob_rf)

plt.figure(figsize=(7,5))

plt.plot(fpr, tpr, label='Random Forest')
```

```
plt.plot([0,1], [0,1], linestyle='--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")

plt.legend()
plt.show()

#
# 19. Feature Importance
#

feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
})

feature_importance = feature_importance.sort_values(
```

```
    by='Importance',
    ascending=False
)

print("\nTop Feature Importance:")
print(feature_importance.head(10))

# Plot Feature Importance

plt.figure(figsize=(10,6))

sns.barplot(
    x='Importance',
    y='Feature',
    data=feature_importance.head(10)
)

plt.title("Top 10 Important Features")
plt.show()

#
==========
```

```
# 20. Risk Scoring System
#
================================================================


risk_probabilities = rf_model.predict_proba(X_test)[:,1]


risk_df = pd.DataFrame({
    'Actual': y_test.values,
    'RiskProbability': risk_probabilities
})


# Risk Categorization


def categorize_risk(prob):


    if prob <= 0.30:
        return "Low Risk"


    elif prob <= 0.60:
        return "Medium Risk"


    else:
```

```
    return "High Risk"

risk_df['RiskCategory'] = risk_df[
    'RiskProbability'
].apply(categorize_risk)

print("\nRisk Scoring Table:")
print(risk_df.head(20))

#
# 21. Sample Prediction
#

sample_customer = X_test.iloc[[0]]

sample_prediction = rf_model.predict(sample_customer)

sample_probability = rf_model.predict_proba(
    sample_customer
)[:,1]
```

```
print("\nSample Prediction:")

print("Prediction:", sample_prediction[0])

print("Churn Probability:", sample_probability[0])


#
==========================================================

# 22. Save Model


import joblib

joblib.dump(rf_model, "bank_churn_model.pkl")

print("\nModel Saved Successfully!")
```

 # END OF PROJECT